BACHELOR OF TECHNOLOGY

IN ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

PROJECT REPORT

Ink in the Clouds

An Online Bookstore for Physical Books, eBooks, and Audiobooks

Submitted by:

Kavitha Haima Kidambi – 2411CS020689

Niharika Koyya – 2411CS020328

Naveena Katakam – 2411CS020277

Navya Sri – 2411CS020454

Course / Program: B.Tech CSE-AI&ML

Batch Number: 27

***Guide**: Mr. Vineeth A*

Department of CSE-AI&ML, School of Engineering

Malla Reddy University

Tech Stack: Spring Boot • React • MySQL

1. Problem Statement & Objective

1.1 Background

Independent and small-to-mid-sized bookstores typically rely on manual or fragmented systems to manage their catalog, inventory, and sales — spreadsheets for stock counts, separate point-of-sale software for in-store purchases, and no unified way to sell digital content alongside physical stock. This creates several recurring problems:

- No centralized, browsable catalog that customers can search, filter by category or format, or purchase from online.
- Manual inventory tracking leads to overselling physical stock or delayed updates.
- Physical and digital sales channels are disconnected — eBooks and audiobooks require a separate platform, separate payments, and separate order tracking.
- No self-service admin tools for staff to add titles, adjust prices, or monitor orders in real time.
- No unified order history for returning customers.

1.2 Problem Statement

There is a need for a single, unified web application that lets a bookstore list and sell books in multiple formats — physical (shipped) copies, eBooks (downloadable), and audiobooks (downloadable) — from one catalog, while giving customers a smooth browsing-to-checkout experience and giving administrators the tools to manage catalog, inventory, and orders in real time.

1.3 Objective

The objective of this project, Ink in the Clouds, is to design and build a full-stack online bookstore that:

- Presents a single catalog where each title can exist as a physical book, eBook, or audiobook, each with its own price and availability.
- Automatically manages stock for physical copies (checking and decrementing at checkout) and grants instant digital access for eBook and audiobook purchases upon confirmed payment.
- Provides secure user authentication and role-based access (customer vs. admin) using Spring Security and JWT tokens.
- Offers cart, checkout with simulated payment, order history, and a digital download/stream endpoint gated on purchase confirmation.
- Gives administrators a panel to manage the catalog (create, edit, delete books), monitor all orders across all users, and update order statuses.

2. Tech Stack & Justification

Ink in the Clouds is built as a decoupled 3-tier application: a React single-page frontend, a Spring Boot REST API, and a MySQL relational database.

Layer	Technology	Why it was chosen
Frontend	React + React Router + Axios + Context API	Component-based architecture keeps the catalog, cart, and checkout UI modular and reusable. Context API handles global auth and cart state without the overhead of Redux, which is overkill for this scope. Axios provides a single interceptor point for attaching the JWT to every outgoing request.
Backend	Spring Boot (Java) + Spring Security	Mature, production-proven ecosystem for REST APIs with strong conventions (dependency injection, layered controller-service-repository architecture). Spring Security integrates cleanly for JWT-based authentication and role-based authorization — admin endpoints are locked at the framework level, not with manual if-checks in controllers.
Database	MySQL	The data is inherently relational — users, orders, and order line items have clear one-to-many relationships with referential integrity requirements. MySQL's ACID transactions are critical for checkout, where stock deduction and order creation must succeed or fail together atomically.
Authentication	JWT (JSON Web Tokens) + BCrypt	Stateless tokens allow the API to authenticate requests without server-side session storage. BCrypt is used to hash passwords before storage — plaintext is never persisted or logged at any point.
File Storage	Local filesystem (configurable path)	eBook and audiobook files are stored on the server filesystem at a configurable path (app.file.storage-path). The file path is never exposed publicly — all downloads are gated through the /books/{id}/download endpoint, which verifies purchase status before streaming the file.
Build Tooling	Maven (backend), Vite + npm (frontend)	Standard, well-documented tooling for dependency management and builds in the Spring Boot and React ecosystems respectively.

2.1 Why not a NoSQL database?

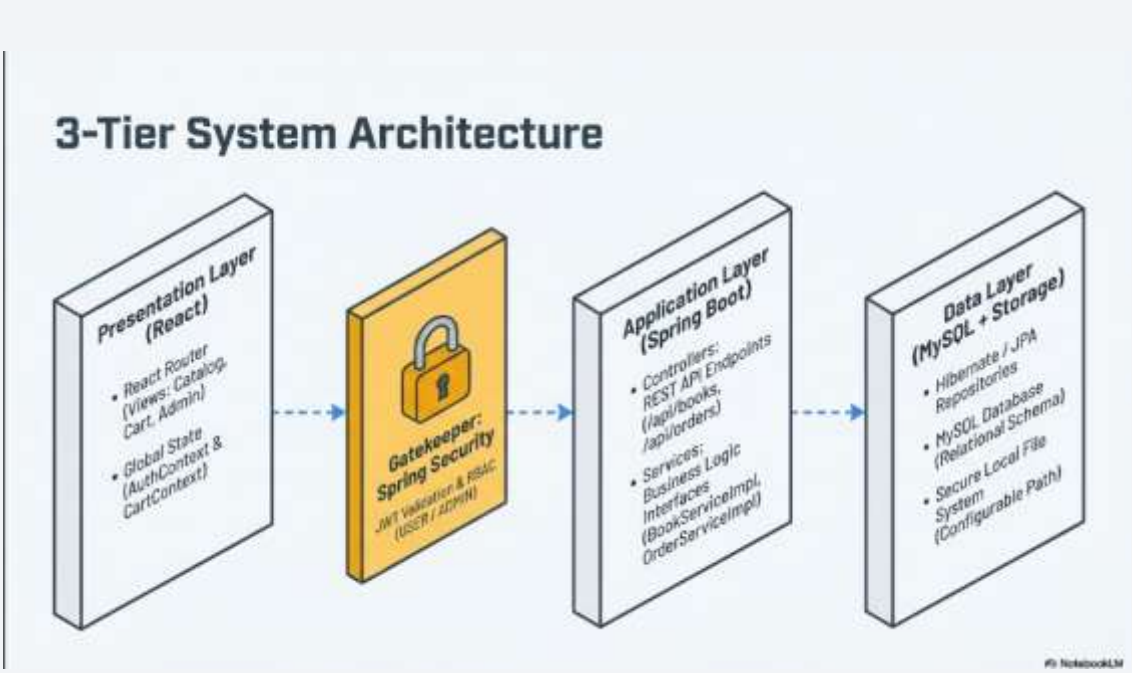
The core entities — users, books, categories, orders, order items — have well-defined relationships and benefit from foreign-key constraints and transactional integrity. An order must not be marked CONFIRMED while its stock deduction fails; a cart item must not reference a non-existent book. A relational database enforces this consistency more naturally than a document store, and the data model does not require the schema flexibility that NoSQL provides.

2.2 Service Layer Pattern

Every service is implemented as an interface plus an implementation class (e.g., BookService interface, BookServiceImpl class). Controllers depend on the interface type, not the concrete class. This keeps business logic independently testable and mockable, and prevents the common anti-pattern of putting business logic directly in controller methods.

3. System Architecture

Ink in the Clouds follows a standard 3-tier architecture, separating presentation, application logic, and data storage so each layer can be developed, tested, and extended independently.



[Figure 1: System Architecture Diagram — 3-tier flow: React frontend ↔ Spring Boot REST API ↔ MySQL]

3.1 Presentation Tier

A React single-page application (SPA) running in the browser. It renders the catalog, book detail pages, cart, checkout flow, order history, and admin panel. All communication with the backend is via HTTPS REST calls using Axios. The JWT access token is stored in localStorage and attached to every outgoing request via an Axios interceptor. On app load, the token is validated by calling GET /auth/me to rehydrate the user session — if the token is expired or invalid, it is cleared and the user is treated as logged out.

3.2 Application Tier

A Spring Boot REST API organized into four layers:

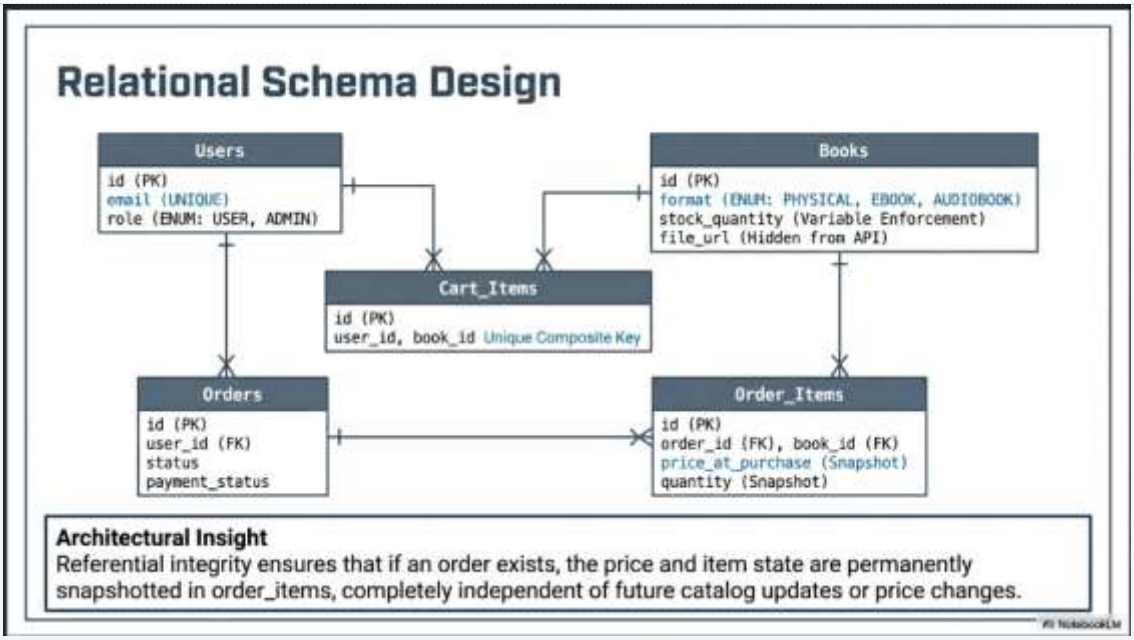
- Controller layer — receives HTTP requests, validates input via Bean Validation, maps to DTOs, and delegates to the service layer.
- Service layer — contains all business logic: stock validation, order creation, payment simulation, purchase-gating for digital downloads.
- Repository layer — Spring Data JPA interfaces providing database access. Custom JPQL queries handle search/filter, atomic stock decrement/increment, and purchase-status checks.
- Security layer — a JWT filter intercepts every request, validates the token, and populates the Spring Security context. Role-based access rules are enforced at the framework level (admin endpoints require ROLE_ADMIN; cart and order endpoints require any authenticated user; catalog browsing is public).

3.3 Data Tier

A MySQL database stores all application data. Hibernate/JPA manages the ORM mapping from Java entities to tables. The schema uses ddl-auto=update in development, meaning Hibernate auto-creates and updates tables from entity definitions. Digital asset files (eBook and audiobook files) are stored on the server filesystem, with only the filename stored in the database — the full path is resolved server-side and the file is never exposed as a public static URL.

4. Database Schema

The schema is straightforward and relational. The key design decision is that a book's format (physical, ebook, or audiobook) is stored as an enum column on the books table itself — not as a separate formats table. This simplifies the data model while still supporting the different fulfillment rules (stock tracking vs. digital delivery) through application-level logic.



[Figure 2: ER Diagram — entities: users, categories, books, cart_items, orders, order_items]

4.1 Table Reference

Table	Key Columns	Purpose
users	id (PK), name, email (UNIQUE), password_hash, role (USER/ADMIN), created_at	Stores customer and admin accounts. Implements Spring Security's UserDetails directly. Passwords are stored as BCrypt hashes — never plaintext.
categories	id (PK), name (UNIQUE)	Genre/category taxonomy (e.g., Fiction, Sci-Fi, Non-Fiction). Used for catalog filtering.
books	id (PK), title, author, isbn, description, price, stock_quantity, category_id (FK), image_url, format (PHYSICAL/EBOOK/AUDIOBOOK), file_url, created_at	One row per book listing. stock_quantity is only meaningful when format = PHYSICAL — it is never checked or decremented for digital formats. file_url stores only the filename (e.g., book.epub), resolved against app.file.storage-path on the server — never sent to the client directly.
cart_items	id (PK), user_id (FK), book_id (FK), quantity, UNIQUE(user_id, book_id)	One row per (user, book) pair. For physical books, quantity is adjustable. For EBOOK/AUDIOBOOK, quantity is always locked to 1 by the backend — the backend overrides any quantity value sent by the client.

Table	Key Columns	Purpose
orders	id (PK), user_id (FK), total_amount, status (PENDING/CONFIRMED/SHIPPED/DELIVERED/CANCELLED), payment_status (PENDING/PAID/FAILED), shipping_address, created_at	One row per checkout. total_amount is snapshotted at order creation and never recomputed. status and payment_status are tracked independently — an order starts as PENDING/PENDING, moves to CONFIRMED/PAID on successful payment, or CANCELLED/FAILED if payment fails.
order_items	id (PK), order_id (FK), book_id (FK), quantity, price_at_purchase	Line items of an order. price_at_purchase snapshots the book price at order time so later price changes do not affect past orders.

5. Feature List

5.1 Customer-Facing Features (Implemented)

- User registration and login with JWT-based authentication. Passwords hashed with BCrypt.
- Profile management — update name, email, and password via PUT /auth/me. A fresh JWT is issued on every successful update to avoid session invalidation.
- Browse catalog with search (by title or author), category filter, and format filter (Physical / eBook / Audiobook). Paginated results (20 per page).
- Book detail page showing format badge (Physical / eBook / Audiobook), price, stock status, and category.
- Cart — add, update quantity (physical only), and remove items. Digital items are always quantity 1, with no quantity control rendered. Cart total computed server-side on every mutation.
- Checkout — converts cart to a PENDING order, reserves (decrements) stock for physical items only, clears the cart. Shipping address is collected only when the cart contains physical items.
- Simulated payment — a separate POST /orders/{id}/payment step that validates card shape and checks against hardcoded test failure cards. On success: order becomes CONFIRMED/PAID, stock stays decremented. On failure: order becomes CANCELLED/FAILED, stock is released back for physical items.
- Digital delivery — GET /books/{id}/download streams the file only if the requesting user has a CONFIRMED or DELIVERED order containing that book. Returns 403 otherwise.
- Purchase status check — GET /books/{id}/purchase-status returns {purchased: true/false}. Used by the frontend to show a Read/Listen button instead of Add to Cart for already-owned digital books.
- Order history — view all past orders with status, payment status, and line item breakdown.
- Related books carousel on the book detail page, showing other titles in the same category.

5.2 Admin-Facing Features (Implemented)

- Full CRUD on books — create, edit, delete. Format-aware form: stock quantity field hidden for digital formats, file URL field hidden for physical formats.
- View all orders across all users, paginated, sorted by newest first.
- Update order status (PENDING → CONFIRMED → SHIPPED → DELIVERED → CANCELLED).

5.3 Stretch Features (Not Implemented)

- Reviews and ratings on books.
- Wishlist.
- Real payment gateway integration (Stripe/Razorpay) — the current payment step is a simulation against the project's own backend. No real money moves, no third-party API calls.
- Book recommendations based on purchase history.

6. API Documentation

All endpoints are prefixed with /api. Every successful response uses the envelope { "data": ..., "message": null }. Every error uses { "error": "SHORT_CODE", "message": "human readable" }. Standard HTTP status codes apply (200, 201, 400, 401, 403, 404, 409, 500).

6.1 Auth

Method	Endpoint	Auth	Description
POST	/auth/register	None	Register a new user. Returns 201 with user object (no password).
POST	/auth/login	None	Authenticate. Returns JWT token + user object.
GET	/auth/me	JWT	Get current authenticated user.
PUT	/auth/me	JWT	Update name, email, password. Returns fresh token + updated user.

6.2 Books

Method	Endpoint	Auth	Description
GET	/books	None	List books. Query params: page, size, search, categoryId, format.
GET	/books/{id}	None	Get single book detail.
POST	/books	ADMIN	Create book. stockQuantity not required for EBOOK/AUDIOBOOK.
PUT	/books/{id}	ADMIN	Update book.
DELETE	/books/{id}	ADMIN	Delete book.
GET	/books/{id}/download	USER	Stream/download digital file. Returns 403 if not purchased, 400 if book is PHYSICAL.
GET	/books/{id}/purchase-status	USER	Returns { purchased: true/false } for the requesting user.
GET	/admin/books/{id}	ADMIN	Get book including fileUrl (for admin edit form).

6.3 Categories

Method	Endpoint	Auth	Description
GET	/categories	None	List all categories.
POST	/categories	ADMIN	Create a category.

6.4 Cart

Method	Endpoint	Auth	Description
GET	/cart	USER	Get current user's cart with items and total.
POST	/cart/items	USER	Add item. For digital books, quantity is always set to 1 regardless of input.
PUT	/cart/items/{id}	USER	Update quantity. Ignored for digital items (locked at 1).
DELETE	/cart/items/{id}	USER	Remove item.
DELETE	/cart	USER	Clear entire cart.

6.5 Orders

Method	Endpoint	Auth	Description
POST	/orders	USER	Checkout. Validates and reserves stock (physical only), clears cart, creates PENDING order.
POST	/orders/{id}/payment	USER	Simulated payment. Body: {cardNumber, expiry, cvv}. Returns 200 with updated order on success or failure; 409 if order is already paid/cancelled.
GET	/orders	USER	Get current user's orders.
GET	/orders/{id}	USER/ADMIN	Get order detail. Users can only view their own; admins can view any.
GET	/admin/orders	ADMIN	All orders, paginated, sorted newest first.
PUT	/admin/orders/{id}/status	ADMIN	Update order status.

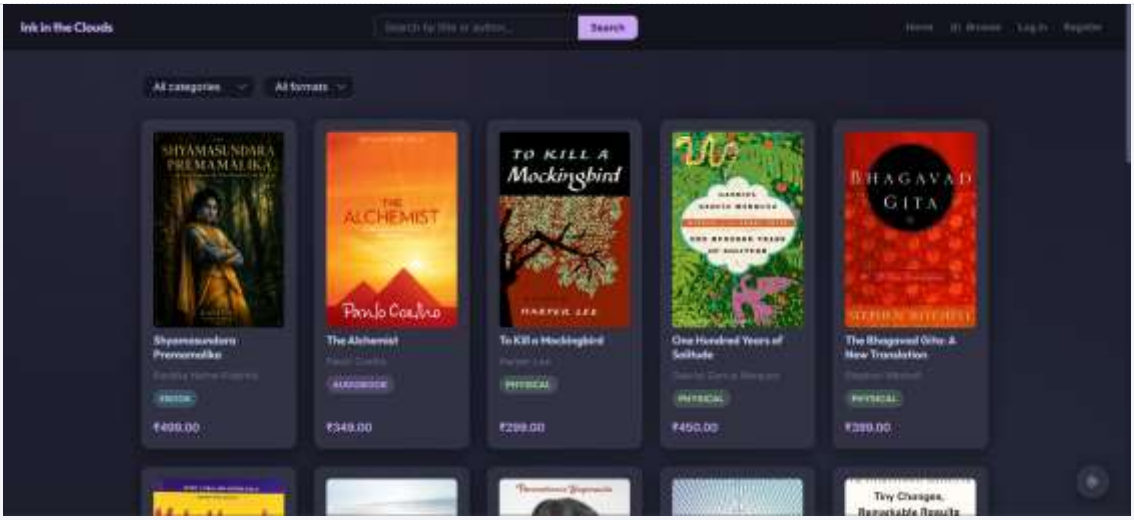
7. Screenshots / UI Walkthrough

The application uses a dark-mode-default design system (Catppuccin Mocha palette) with a light mode toggle. All currency is displayed in Indian Rupees (₹).

7.1 Landing Page

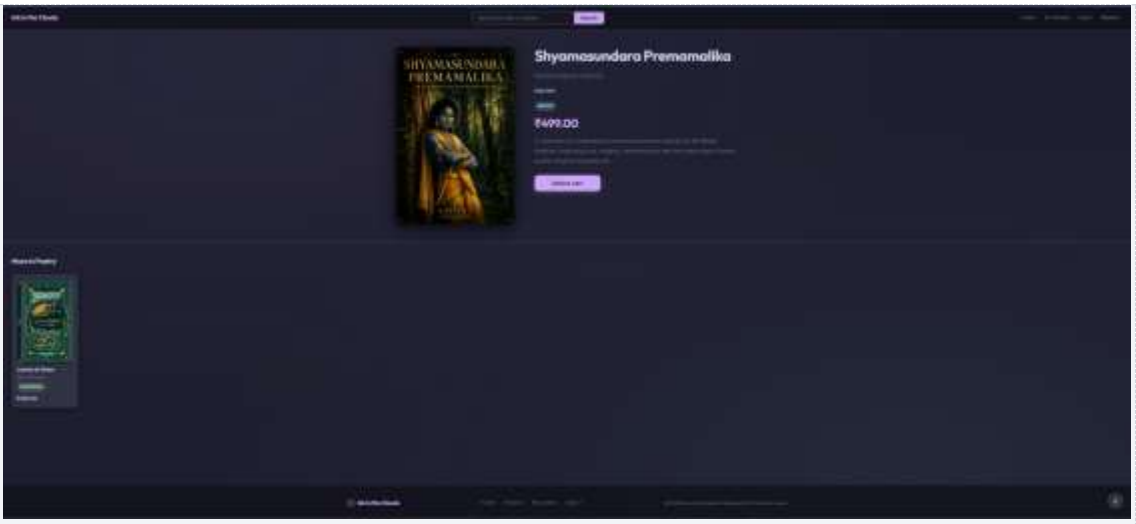
[Screenshot: Landing page — 'Ink in the Clouds' hero text with Browse Books CTA button]

7.2 Catalog / Browse Page



[Screenshot: Catalog page — book grid with search bar, category dropdown, format filter dropdown, pagination]

7.3 Book Detail Page — Physical Book



[Screenshot: Book detail — physical book showing stock pill, Add to Cart button, related books carousel]

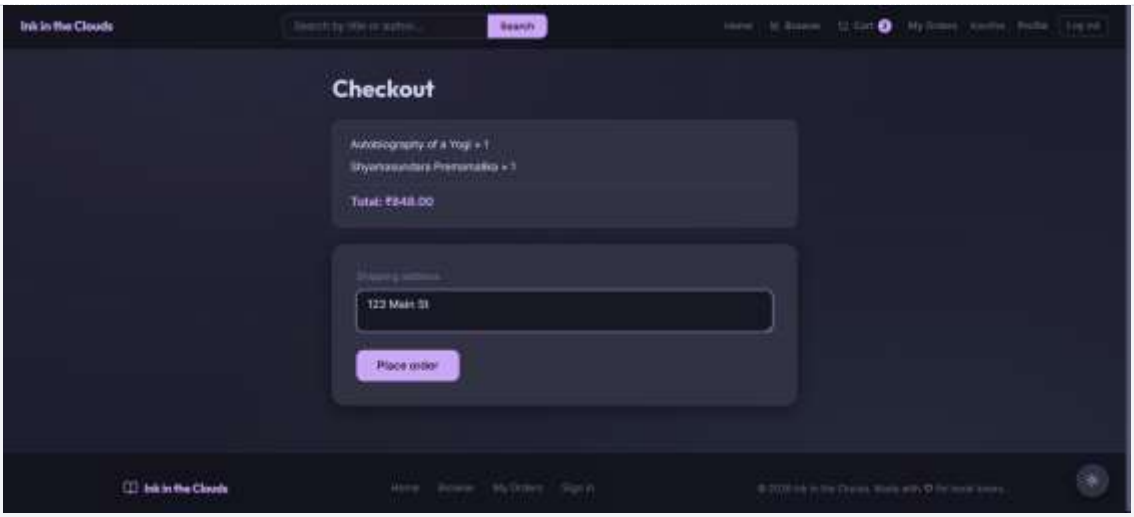
7.4 Book Detail Page — AudioBook (Already Purchased)

[Screenshot: Book detail —AudioBook showing 'Listen' button instead of Add to Cart (purchased state)]

7.5 Cart Page

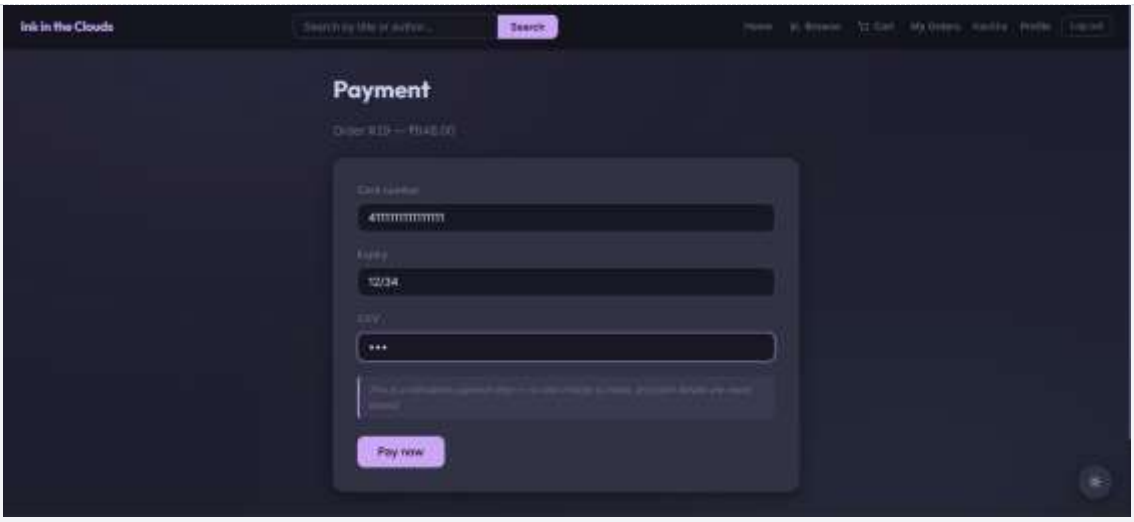
[Screenshot: Cart — mix of physical (with quantity input) and digital (Qty: 1, no input) items, total, checkout button]

7.6 Checkout — Shipping Step



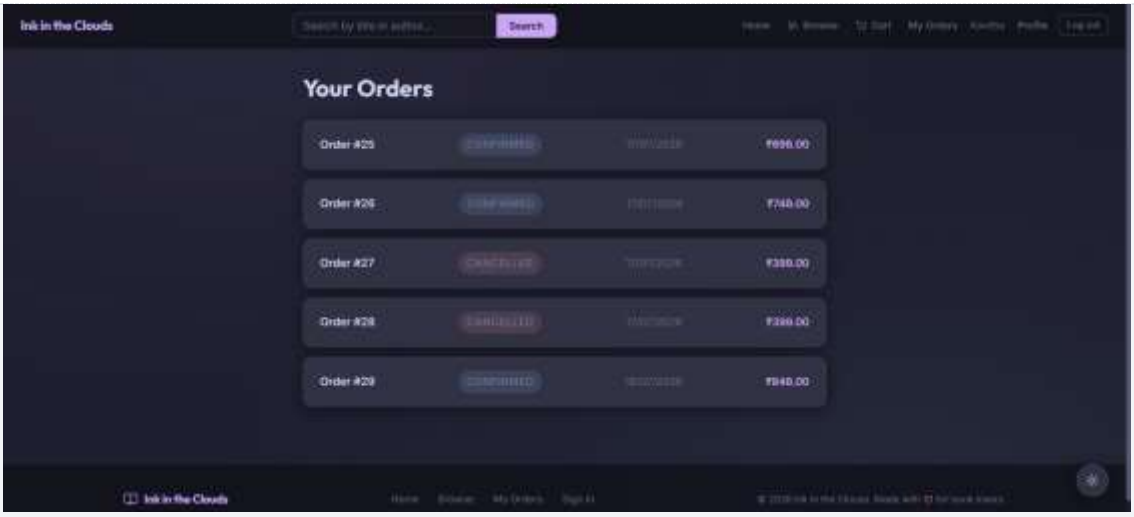
[Screenshot: Checkout shipping address form (shown only for physical items)]

7.7 Checkout — Payment Step



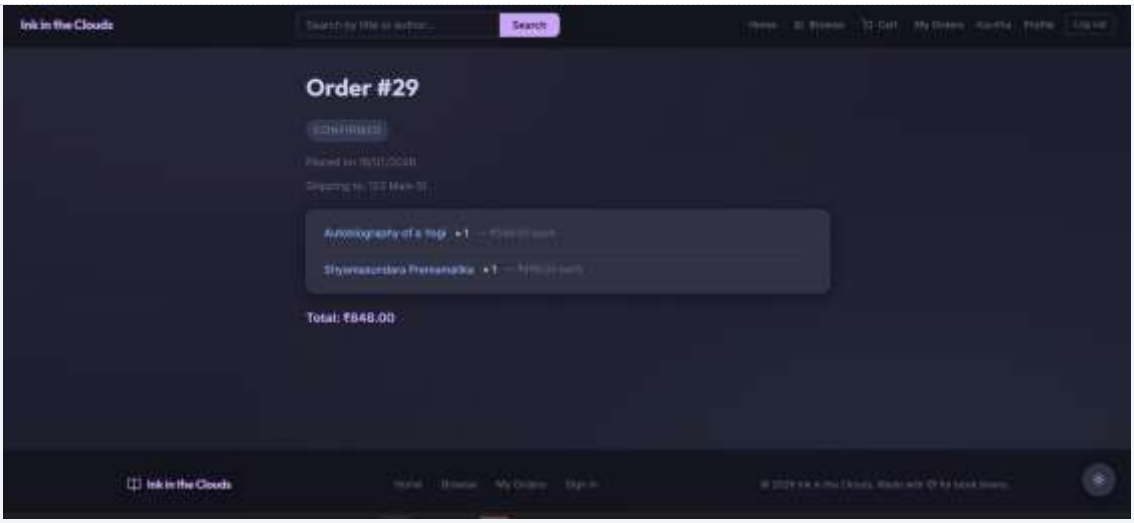
[Screenshot: Payment form with card number, expiry, CVV fields and 'This is a simulated payment' disclaimer]

7.8 Order History



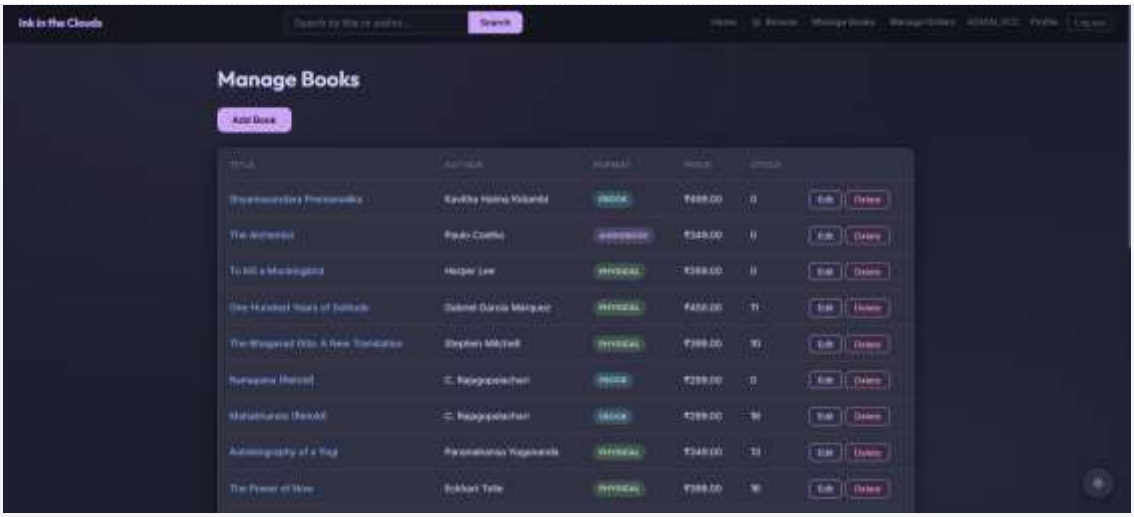
[Screenshot: Orders page — list of order cards with status badges (PENDING/CONFIRMED/CANCELLED etc.)]

7.9 Order Detail



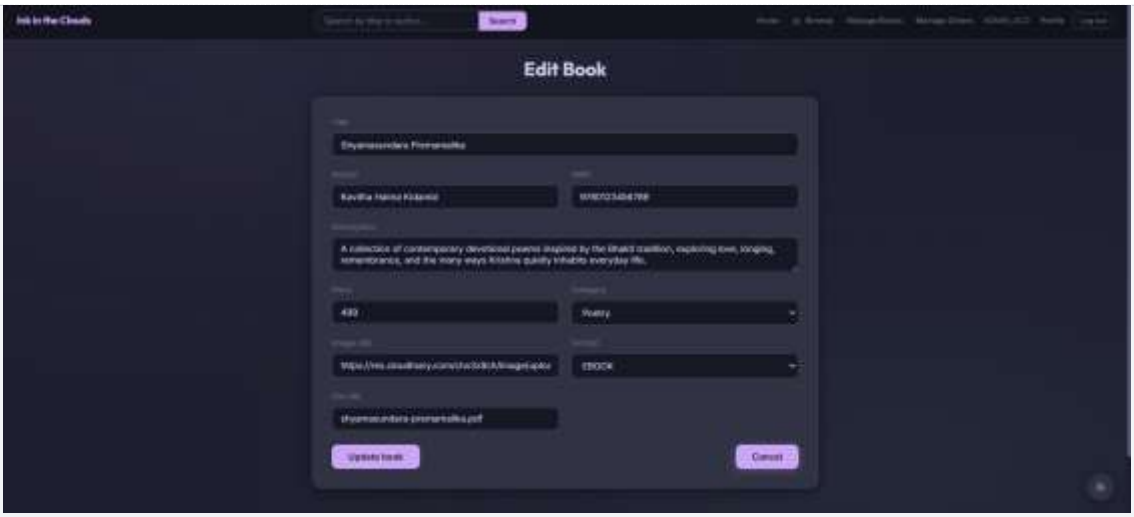
[Screenshot: Single order detail with items, quantities, prices, shipping address, status]

7.10 Admin — Manage Books



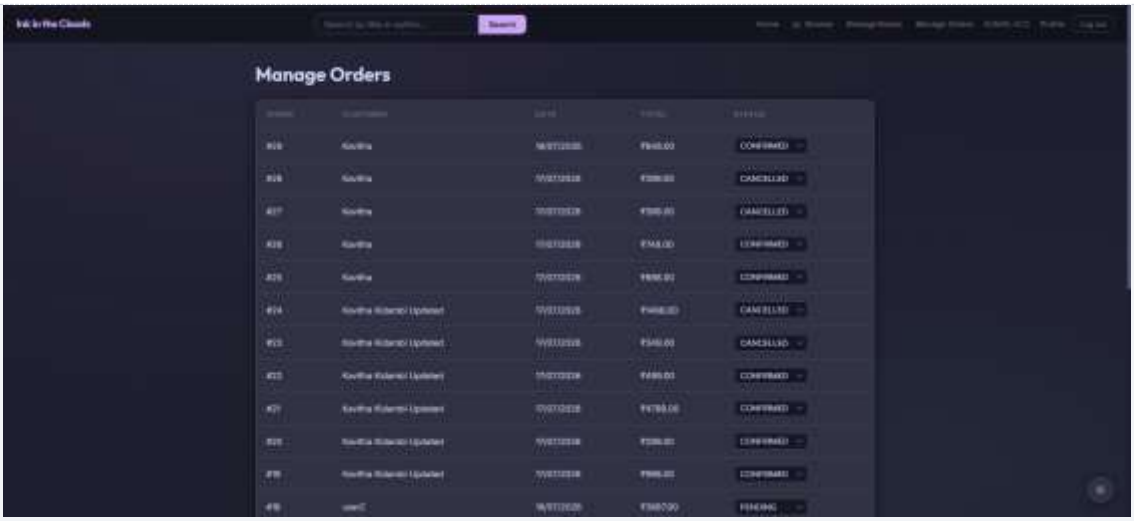
[Screenshot: Admin books table with Edit/Delete buttons, Add Book button, format badges]

7.11 Admin — Book Form



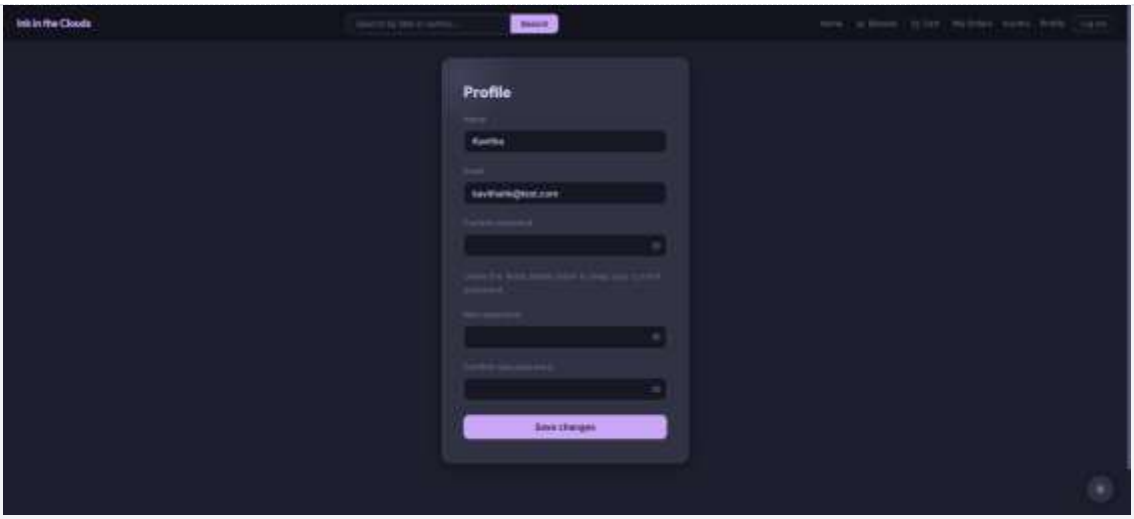
[Screenshot: Admin book creation/edit form — showing format dropdown, file URL field (digital only), stock quantity field (physical only)]

7.12 Admin — Manage Orders



[Screenshot: Admin orders table with status dropdown per row for updating order status]

7.13 Profile Page



[Screenshot: Profile page — name, email, current password, optional new password fields]

10. Challenges Faced & Solutions

10.1 Stock validation race conditions at checkout

Problem: Two users could add the last physical copy of a book to their cart simultaneously. Both checkouts could appear to succeed, resulting in the stock going negative.

Solution: Stock decrement is performed using an atomic conditional UPDATE query: UPDATE books SET stock_quantity = stock_quantity - ? WHERE id = ? AND stock_quantity >= ?. This query returns 0 rows updated if there is insufficient stock at the moment of execution. The service layer checks the return value — if 0, it throws InsufficientStockException and Spring's @Transactional rolls back all decrements that already ran in the same checkout loop, preventing partial reservations.

10.2 Physical vs. digital items in the same order

Problem: Physical and digital items need entirely different checkout behavior — stock checking, stock decrement, stock release on payment failure, shipping address requirement — but they can coexist in the same cart and order.

Solution: Every loop in the checkout and payment service explicitly checks book.getFormat() != BookFormat.PHYSICAL before applying stock operations, and skips digital items entirely. The frontend similarly checks item.book.format to decide whether to render a quantity input or a static 'Qty: 1' label, and whether to show the shipping address form.

10.3 JWT and role changes taking effect immediately

Problem: If a user's role is updated in the database (e.g., promoted to ADMIN), the change should take effect on the very next request — not only after their token expires.

Solution: The JWT contains only the user's email as the subject — no role claim. On every request, JwtAuthFilter extracts the email, calls CustomUserDetailsService.loadUserByUsername() to fetch the current user from the database, and uses that object's authorities for authorization. This means role changes take effect immediately on the next request, at the cost of one extra DB read per request.

11. Future Scope

- Reviews and star ratings on books, visible to all users and factored into catalog sorting.
- Wishlist — save books for later without adding to cart.
- Real payment gateway integration (Razorpay or Stripe) with webhooks for asynchronous payment confirmation, replacing the current simulation.
- Book recommendations based on purchase and browsing history.
- COD (Cash on Delivery) support for physical books — requires a separate fulfillment flow that bypasses the payment endpoint.
- Mobile application (React Native) reusing the existing REST API.
- Admin analytics dashboard — best-selling titles, revenue by format, low-stock alerts.
- Account deletion with appropriate data retention/removal handling.

Conclusion

Ink in the Clouds demonstrates a complete, end-to-end full-stack application built with industry-standard technologies and architecture patterns. The project covers the full lifecycle of a feature: from database schema design through service-layer business logic, REST API contract, and a polished React frontend — including non-trivial concerns such as role-based access control, atomic stock management under concurrent load, simulated payment flows, and purchase-gated digital delivery.

The layered architecture (Controller → Service → Repository) and the interface+implementation pattern used for every service make the codebase independently testable and extensible. The separation between physical and digital fulfillment logic is clean and explicit throughout the stack, making it straightforward to add new formats or fulfillment rules in the future.

The platform successfully unifies what would otherwise require three separate systems — a physical bookstore inventory system, a digital content delivery platform, and an order management dashboard — into a single, maintainable application.